RISC-V VAES Vector Cryptography Coprocessor
Microarchitecture Design and Functional Verification

Project Deliverable	SystemVerilog RTL, self-checking Verilator testbench, design PDF, ISA markdown
Target Instructions	vaes.sbox.v, vaes.srow.v, vaes.mcol.v, vaes.ark.v
Execution Model	Normal 5-stage pipeline: IF / ID / EX / MEM / WB
Verification Environment	Verilator with testbench reference model, directed tests, randomized tests, and SVA

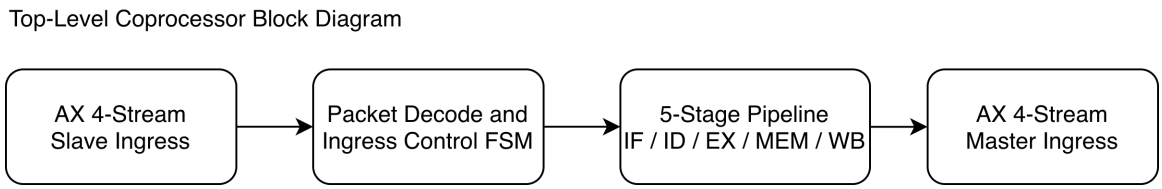

Fig. 1. Top-level block diagram of the coprocessor and its verification boundaries.

1. Introduction

This design implements a dedicated AES-oriented vector coprocessor for RISC-V using four custom vector instructions. The datapath uses a normal five-stage pipeline and a 32-entry, 128-bit vector register file. The coprocessor communicates with the host through AXI4-Stream, where the input stream carries instruction and data packets and the output stream returns 128-bit results.

The verification environment is built for Verilator. A SystemVerilog class-based bit-accurate reference model is used only inside the testbench and scoreboard. It is not reused as any RTL pipeline stage. The regression includes a NIST known-answer test, directed variable-VL tests, randomized AES encryptions, randomized instruction streams, output backpressure, and SVA checks for both interface and internal control legality.

2. Design targets and assumptions

- Four required custom vector instructions are implemented exactly: vaes.sbox.v, vaes.srow.v, vaes.mcol.v, and vaes.ark.v.
- The vector register file contains 32 registers, each 128 bits wide.
- The pipeline is a normal five-stage IF / ID / EX / MEM / WB structure. Register read is folded into the ID stage; there is no separate RR stage.
- Dedicated parallel SubBytes and MixColumns units exist in EX, and both are modeled as fixed multi-cycle operations.
- The legal vector lengths are 4, 8, 12, and 16 bytes so that AES column granularity is preserved.
- The reference model remains strictly inside the Verilator testbench. It is used for sequence generation, state mirroring, and scoreboard comparisons only.

3. Interface definition

The coprocessor exposes one AXI4-Stream slave input and one AXI4-Stream master output. The slave stream carries either instruction packets or auxiliary control/data packets.

3.1 AXI4-Stream ingress

Signal	Width	Description
s_axis_tvalid	1	Input packet valid
s_axis_tready	1	Coprocessor ready to accept current packet
s_axis_tdata	256	Tagged instruction/data packet
s_axis_tlast	1	Always asserted for one-packet transactions

3.2 AXI4-Stream egress

Signal	Width	Description
m_axis_tvalid	1	Result packet valid
m_axis_tready	1	Downstream sink ready
m_axis_tdata	128	Returned vector result
m_axis_tlast	1	Always asserted with each result packet

3.3 Ingress packet format

Bits	Field	Meaning	Used by
255:248	pkt_type	Packet kind	LOAD / INST / CFG / STORE
247:216	word0	Raw 32-bit custom instruction word	INST
215:211	reg_idx	Vector register index	LOAD / STORE
210:206	vl	Vector length in bytes	CFG
205:128	reserved	Reserved and set to zero	All
127:0	payload	128-bit vector data	LOAD

4. Custom instruction set

All four custom instructions share the same 32-bit custom R-type format: funct6 | vm | vs2 | vs1 | funct3 | vd | opcode. The fixed fields are opcode = custom-0 (0001011), funct3 = 000, and vm = 1.

Mnemonic	funct6	Operands	Function	Latency
vaes.sbox.v	000000	vd, vs1	Apply AES S-box to each active byte	2 cycles
vaes.srow.v	000001	vd, vs1	Apply ShiftRows on active AES columns	1 cycle
vaes.mcol.v	000010	vd, vs1	Apply MixColumns on active AES columns	2 cycles
vaes.ark.v	000011	vd, vs1, vs2	XOR active bytes with round key	1 cycle

The full bit-level instruction definition is provided in the companion file `riscv_vaes_extension.md`. That file is the normative ISA specification for the deliverable.

5. Microarchitecture

The core is organized around a single instruction stream and a centralized vector register file. LOAD, CFG, and STORE packets are treated as control-plane operations on the AXI ingress. The four custom VAES instructions flow through the five-stage instruction pipeline.

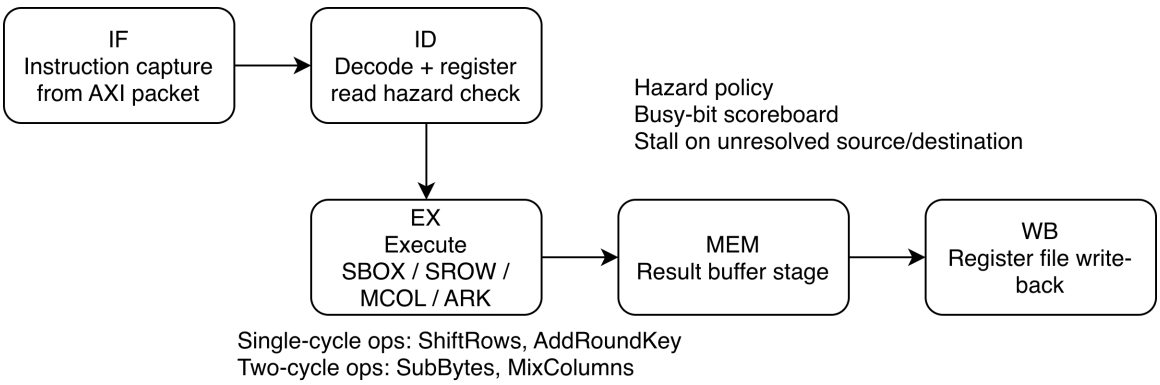

Fig. 2. Five-stage pipeline and multi-cycle execution behavior.

5.1 Stage-by-stage behavior

- IF captures one raw 32-bit custom instruction from the ingress packet stream and snapshots the current VL.
- ID decodes the instruction, reads the vector sources, and applies scoreboard-based hazard checking.
- EX selects one of the four execution units. ShiftRows and AddRoundKey are single-cycle. SubBytes and MixColumns are modeled as fixed 2-cycle operations.
- MEM acts as a result holding stage so the five-stage structure remains explicit and normal.
- WB writes the destination register and clears the corresponding busy bit.

5.2 Execute-stage hardware

The execute stage instantiates four dedicated combinational units: `vaes_sbox_unit`, `vaes_srow_unit`, `vaes_mcol_unit`, and `vaes_ark_unit`. The SubBytes unit performs 16 parallel byte substitutions. The MixColumns unit computes all active AES columns in parallel. A lightweight control FSM keeps SubBytes and MixColumns in EX for two cycles before forwarding the result to MEM.

5.3 Hazard handling

A 32-bit busy-bit scoreboard tracks unresolved register destinations. The ID stage stalls when a source register is still busy or when a younger instruction would overwrite a destination that is already pending. This conservatively handles RAW and WAW hazards. Because the regression is correctness-driven rather than throughput-driven, no bypass network is used.

5.4 AXI handshake control

The ingress logic accepts instruction packets only when the IF stage is empty. LOAD/CFG/STORE packets are accepted only when the pipeline is empty and no egress result is pending. The result channel holds `m_axis_tvalid` until `m_axis_tready` is observed. SVA checks ensure the output payload remains stable while stalled.

6. Key RTL modules

Module	Responsibility
vaes_common_pkg.sv	Shared constants, enums, instruction encoding/decoding helpers, and packet constructors.
vaes_aes_pkg.sv	Synthesizable AES helper functions used by the execution units.
vaes_regfile.sv	32 x 128-bit vector register file with one write port and two asynchronous read ports.
vaes_sbox_unit.sv	Parallel SubBytes datapath.
vaes_srow_unit.sv	ShiftRows datapath.
vaes_mcol_unit.sv	Parallel MixColumns datapath.
vaes_ark_unit.sv	AddRoundKey datapath.
vaes_coproc_top.sv	Top-level pipeline, AXI handshake logic, scoreboard, and control FSM.

7. Verification architecture

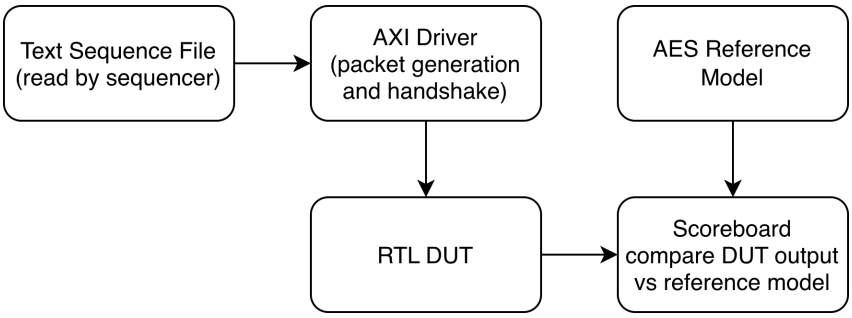

Fig. 3. Verification flow used by the Verilator testbench.

The testbench is pure SystemVerilog and targets Verilator. The stimulus source is a text sequence file that is parsed by the sequencer. Each parsed command is translated into an AXI4-Stream packet for the DUT. At the same time, the reference model mirrors the architectural state and predicts expected STORE results, which are queued for scoreboard comparison.

7.1 Reference model usage

The `vaes_ref_model` class provides a bit-accurate AES-128 model, key expansion, and architectural-state mirroring for the four custom instructions. It is never instantiated inside the RTL and it is not used as a pipeline stage. This separation keeps the implementation independent while still providing strong self-checking capability.

7.2 Directed tests

- NIST AES-128 known-answer test using plaintext `00112233445566778899aabbccddeeff` and key `000102030405060708090a0b0c0d0e0f`.
- Directed variable-VL tests at 4, 8, 12, and 16 bytes to exercise partial-width execution semantics.
- Hazard-heavy instruction streams that repeatedly overwrite the same state register.

7.3 Randomized tests

To remain Verilator-friendly out of the box, the randomized test generator uses solver-free constrained generation rather than relying on external SMT-based `randomize()` support. The generator writes random text sequence files and then re-reads them through the exact same sequencer path. Two random families are included:

- Random AES tests: random plaintext/key pairs, full-length VL = 16, full AES-128 round schedule.
- Random ISA tests: random VL chosen from {4, 8, 12, 16}, random LOADs, random instruction mixes, random STORE points, and randomized output backpressure.

7.4 Assertions

- Ingress packets must assert `TLAST` when accepted.
- The output channel must hold `TVALID`, `TDATA`, and `TLAST` stable while stalled.
- Control-state assertions tie `CTRL_EX_SBOX`, `CTRL_EX_MCOL`, and `CTRL_EGRESS` to legal internal conditions.
- Write-back is asserted only for destinations that are marked busy by the scoreboard.

8. Regression result used for sign-off

The package was compiled and executed with Verilator 5.38 in the preparation environment used for this deliverable. The shipped regression completed successfully with:

- 22 tests completed
- 65 output packets matched against the reference model
- Zero scoreboard mismatches
- Zero simulation runtime errors

9. How to run

The package includes a ready-to-use shell script and a Makefile target. The recommended command is `make run`. The script falls back to `python -m verilator` if the Verilator Python wrapper is used instead of a system `verilator` executable.

10. Delivered files

- `rtl/` — synthesizable SystemVerilog source
- `tb/` — self-checking SystemVerilog testbench, reference model, and example text sequences
- `riscv_vaes_extension.md` — ISA definition
- `docs/vaes_coproc_design.pdf` — this design specification
- `sim/run_verilator.sh` and `Makefile` — reproducible simulation entry points